



**Narendra
Pratap Singh**
DevOps & Cloud Engineer

VERSION RELATED THINGS

THE COMPLETE GUIDE FOR DEVOPS ENGINEERS

In DevOps, everything is versioned: code, releases, artifacts, infrastructure.

Mastering versioning ensures traceability, consistency, and reliable deployments.



linkedin.com/in/narendra09-devops



github.com/narendra09-devops



DevOps Learning Hub

https://narendra09-devops.github.io/devops-learning-hub/

1 VERSION CONTROL – Track Code Changes and Collaborate Efficiently

WHAT TO LEARN

- What is Version Control & why it matters
- Git Basics: repo, commit, branch, merge
- Branching Strategies (Git Flow, GitHub Flow, Trunk Based)
- Merge, Rebase, Cherry-pick, Stash
- Resolve Merge Conflicts
- Pull Requests & Code Reviews
- Git Hooks & Branch Protection
- Git Best Practices

KEY TOPICS

- Working Directory, Staging Area, Repository, Commit History
- Branching & Merging
- Remote Repositories
- Forking & Cloning
- Pull Requests Workflow
- Code Review & Collaboration

COMMON COMMANDS

```
git init
git clone <repo>
git status
git add .
git commit -m "message"
git push origin <branch>
git pull
git branch
git checkout -b feature/xyz
git merge feature/xyz
git rebase main
git stash
```

TOOLS

- Git
- GitHub
- GitLab
- Bitbucket

BEST PRACTICES

- ✓ Commit small, meaningful changes
- ✓ Write clear commit messages
- ✓ Use branches for new features
- ✓ Always review before merge
- ✓ Protect main/master branch

2 RELEASE VERSIONING – Manage Application Releases Properly

WHAT TO LEARN

- Semantic Versioning (MAJOR.MINOR.PATCH)
- Git Tags & Releases
- Changelog & Release Notes
- Rollback Strategy
- Hotfix Releases
- Pre-release vs Stable Releases

KEY TOPICS

- Semantic Versioning Rules
- Git Tagging
- GitHub/GitLab Releases
- CHANGELOG.md
- Release Workflow
- Rollback & Hotfix

EXAMPLE – SEMANTIC VERSIONING

MAJOR

1

Breaking
Changes

MINOR

2

New Features
Backward
Compatible

PATCH

3

Bug Fixes
Backward
Compatible

BEST PRACTICES

- ✓ Follow Semantic Versioning strictly
- ✓ Tag every production release
- ✓ Maintain CHANGELOG.md
- ✓ Use meaningful release notes
- ✓ Plan rollback before release

TOOLS



Git Tags



GitHub Releases



GitLab Releases

EXAMPLE COMMANDS

```
git tag v1.2.3
git push origin v1.2.3
git tag -a v1.2.3 -m "Release v1.2.3"
```

3 ARTIFACT VERSIONING – Version Your Build Artifacts and Images

WHAT TO LEARN

- What are Artifacts?
- Docker Image Versioning
- Artifact Repositories
- Immutable vs Mutable Artifacts
- Traceability from Source to Artifact

KEY TOPICS

- Docker Image Tags
- Artifact Repositories (Nexus, JFrog Artifactory, GitHub Packages)
- Immutability & Traceability
- Retention Policies

EXAMPLES

Docker Image Tagging

```
docker build -t myapp:v1.2.3 .
docker tag myapp:v1.2.3 user/myapp:v1.2.3
docker push user/myapp:v1.2.3
```

Common Tags

v1.2.3 latest dev staging prod commit-sha

BEST PRACTICES

- ✓ Never use only latest in production
- ✓ Use immutable versioned tags
- ✓ Store artifacts in repository
- ✓ Scan artifacts for vulnerabilities
- ✓ Keep retention & cleanup policy

TOOLS



Docker



Docker Hub



Nexus



JFrog Artifactory



GitHub Packages



AWS ECR



Azure ACR



Google Artifact Registry

4 INFRASTRUCTURE VERSIONING – Version Your Infrastructure as Code

WHAT TO LEARN

- Why version infrastructure?
- Terraform Versioning
- Module Versioning
- Ansible Versioning
- Helm Chart Versioning
- Kubernetes Manifest Versioning

KEY TOPICS

- Terraform Version & Provider Version
- Terraform Remote State Versioning
- Ansible Playbook & Role Versioning
- Helm Chart Version & AppVersion
- Kubernetes Deployments & Rollbacks

EXAMPLES

Terraform Version

```
terraform {
  required_version = ">= 1.5.0"
}
```

Module Version

```
module "vpc" {
  source = "github.com/user/vpc?ref=v1.0.0"
}
```

TOOLS

- Terraform
- Ansible
- Helm
- Kubernetes

BEST PRACTICES

- ✓ Version your IaC in Git
- ✓ Use module versioning
- ✓ Pin provider versions
- ✓ Use Helm chart versioning
- ✓ Keep rollback strategy

5 END-TO-END VERSION FLOW IN DEVOPS



Every step should be versioned and traceable!

GOLDEN RULES OF VERSIONING

- ★ Everything should be versioned
- ★ Every release should be traceable
- ★ Never deploy unknown code
- ★ Use immutable artifacts
- ★ Plan rollback before rollout



Keep Learning. Keep Building. Let's connect and grow together!



linkedin.com/in/narendra09-devops



github.com/narendra09-devops



DevOps Learning Hub

https://narendra09-devops.github.io/devops-learning-hub/



**Narendra
Pratap Singh**
DevOps & Cloud Engineer

Narendra Pratap Singh

DevOps & Cloud Engineer

🎯 Sharing knowledge | Building in Public | Growing Together

Let's Connect:



linkedin.com/in/narendra09-devops



github.com/narendra09-devops

VERSION RELATED THINGS EVERY DEVOPS ENGINEER MUST MASTER

In DevOps, everything is versioned: code, releases, artifacts, infrastructure.
Mastering versioning ensures traceability, consistency, and reliable deployments.



1 VERSION CONTROL

Track code changes and collaborate efficiently



- Tool: Git (GitHub, GitLab, Bitbucket)
- Branching Strategy (Git Flow / GitHub Flow)
- Commits, Pull Requests, Code Reviews
- Tags for stable releases (v1.0.0, v1.1.0, v2.0.0)
- Semantic Versioning (MAJOR.MINOR.PATCH)

2 RELEASE VERSIONING

Manage application releases properly



- Use Git Tags for releases
- Maintain CHANGELOG.md and RELEASE NOTES
- Follow Semantic Versioning
- Example: v1.0.0 -> v1.0.1 -> v1.1.0 -> v2.0.0
- Rollback with previous stable release

3 ARTIFACT VERSIONING

Version your build artifacts and images



- Docker Image Tags (v1.0.0, latest, staging)
- Artifact Repositories: Docker Hub, ECR, Nexus, JFrog Artifactory
- Immutable Artifacts
- Trace every artifact to source commit

4 INFRASTRUCTURE VERSIONING

Version your infrastructure as code



- Terraform version & provider version
- Module versioning (ref: v1.0.0)
- Ansible, Helm Chart versioning
- Kubernetes manifests with image versions
- Ensures consistent & reproducible environments



BEST PRACTICES

- ✓ Always tag every production release
- ✓ Never use 'latest' in production
- ✓ Keep changelog up-to-date

- ✓ Make every deployment traceable
- ✓ Automate versioning in CI/CD pipelines

- ✓ Rollback plan should be ready
- ✓ Protect main branch & use pull requests



PRACTICE. BUILD. DEPLOY. REPEAT.

Explore my DevOps Learning Hub – a complete roadmap with projects, notes, tutorials, and real-world resources.

<https://narendra09-devops.github.io/devops-learning-hub/>

DevOps Learning Hub

Your one-stop place to learn DevOps the right way!

Explore Now →

💡 Keep Learning. Keep Building.
Let's connect and grow together! 🚀



linkedin.com/in/narendra09-devops



github.com/narendra09-devops



Narendra
Pratap Singh

DevOps & Cloud Engineer

VERSION RELATED THINGS WHAT'S NEXT?

We've covered the big 4 areas. Now let's dive deeper into each topic with detailed, topic-wise insights in the upcoming posts.

[in linkedin.com/in/narendra09-devops](https://www.linkedin.com/in/narendra09-devops)

github.com/narendra09-devops

[DevOps Learning Hub
narendra09-devops.github.io/devops-learning-hub/](https://narendra09-devops.github.io/devops-learning-hub/)

UPCOMING DETAILED POSTS (STAY TUNED!)



1. VERSION CONTROL

Track Code Changes and Collaborate Efficiently

- Git Basics & Core Concepts
- Branching Strategies (Git Flow, GitHub Flow, Trunk Based)
- Merge, Rebase, Cherry-pick, Stash
- Pull Requests & Code Reviews
- Git Hooks & Branch Protection
- Best Practices for Teams

Coming Soon



2. RELEASE VERSIONING

Manage Application Releases Properly

- Semantic Versioning Deep Dive
- Git Tags, GitHub/GitLab Releases
- CHANGELOG.md Best Practices
- Release Workflow & Automation
- Rollback & Hotfix Strategy
- Pre-release vs Stable Releases

Coming Soon



3. ARTIFACT VERSIONING

Version Your Build Artifacts and Images

- Docker Image Versioning
- Artifact Repositories (Nexus, JFrog Artifactory, GH Packages)
- Immutable vs Mutable Artifacts
- Traceability & SBOM
- Retention & Cleanup Policies
- Best Practices for Artifacts

Coming Soon



4. INFRASTRUCTURE VERSIONING

Version Your Infrastructure as Code

- Terraform Versioning & Providers
- Ansible Playbook & Role Versioning
- Helm Chart & App Versioning
- Kubernetes Manifest Versioning
- Environment Promotion Strategy
- Drift Detection & Rollback

Coming Soon



GOAL

Every change should be versioned, traceable, and recoverable.



Code Commit



Git Push



Build



Artifact



Deploy



Monitor



Rollback
(if needed)

VERSION EVERYTHING. TRUST EVERY DEPLOYMENT.



STAY CONNECTED

Follow along for in-depth posts, real-world examples, and best practices to level up your DevOps journey!

KEEP LEARNING. KEEP BUILDING. LET'S GROW TOGETHER! 🚀

[in linkedin.com/in/narendra09-devops](https://www.linkedin.com/in/narendra09-devops)

github.com/narendra09-devops

narendra09-devops.github.io/devops-learning-hub/



**Narendra
Pratap Singh**

DevOps & Cloud Engineer

VERSION CONTROL

THE FOUNDATION OF DEVOPS

Track changes, collaborate, and ship better software faster.
Master Git. Master DevOps.

WHAT YOU WILL LEARN

- ✓ Core Concepts
- ✓ Repositories & Architecture
- ✓ Essential Git Commands
- ✓ Branching Strategies
- ✓ Collaboration Workflows
- ✓ Best Practices



linkedin.com/in/narendra09-devops



github.com/narendra09-devops



DevOps Learning Hub

https://narendra09-devops.github.io/devops-learning-hub/

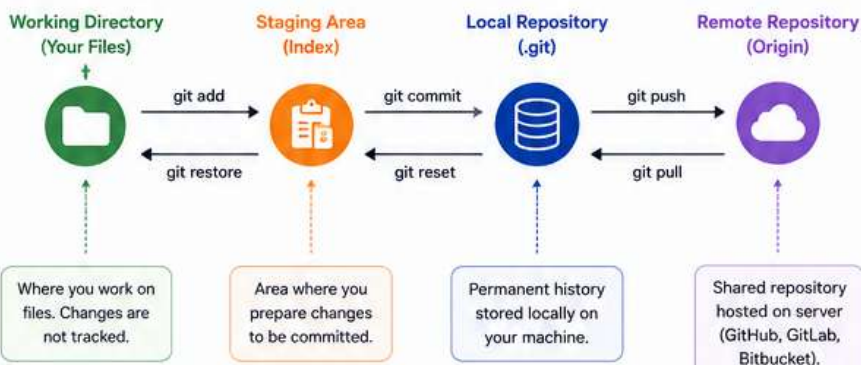
1. WHAT IS VERSION CONTROL?

Version Control System (VCS) records every change to a file or set of files over time so that you can recall specific versions later. It enables multiple developers to work together without overwriting each other's changes.

BENEFITS

- ✓ Track every change and who made it
- ✓ Collaborate with multiple people
- ✓ Branch, experiment & merge safely
- ✓ Revert to previous versions
- ✓ Maintain history and audit trail
- ✓ Backup & disaster recovery

2. GIT ARCHITECTURE



3. MOST USED GIT COMMANDS CHEAT SHEET

Setup & Config	<pre>git config --global user.name "Your Name" git config --global user.email "you@example.com" git config --list</pre>	<pre># Set your name # Set your email # List all config</pre>
Repository	<pre>git init git clone <repo-url> git remote -v</pre>	<pre># Initialize new repo # Clone remote repo # Show remote URLs</pre>
Basic Workflow	<pre>git status git add <file> git add . git commit -m "message" git log --oneline --graph --all</pre>	<pre># Show current status # Stage specific file # Stage all changes # Commit staged changes # View commit history</pre>
Syncing	<pre>git pull origin main git push origin main git fetch</pre>	<pre># Fetch + merge from remote # Push to remote # Fetch without merging</pre>
Branching	<pre>git branch git branch <branch-name> git checkout <branch-name> git switch <branch-name> git checkout -b <branch-name> git branch -d <branch-name> git merge <branch-name></pre>	<pre># List branches # Create new branch # Switch branch (old) # Switch branch (new) # Create & switch # Delete branch # Merge branch</pre>
Undo & Revert	<pre>git restore <file> git restore --staged <file> git reset --soft HEAD~1 git reset --mixed HEAD~1 git reset --hard HEAD~1 git revert <commit-id></pre>	<pre># Discard changes in working dir # Unstage file # Undo last commit (keep changes) # Undo commit (unstage changes) # Undo commit (discard changes) # Create new commit to revert</pre>
Stash	<pre>git stash git stash list git stash apply git stash pop git stash drop</pre>	<pre># Save changes to stash # List stash # Apply latest stash # Apply & remove stash # Delete stash</pre>

Tip: Use `git --help` with any command to get more details.

4. BRANCHING STRATEGIES

Git Flow

Best for projects with scheduled releases.



GitHub Flow

Best for continuous deployment.



Trunk Based Development

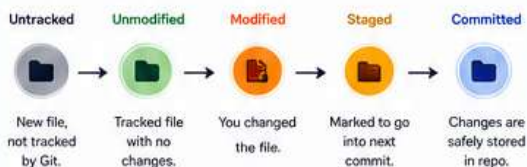
Small changes, feature flags, short-lived branches.



Choose strategy based on your team size, project type, and release cycle.

5. FILE STATES IN GIT

Understanding the lifecycle of your files.



Transitions:

Untracked → Modified	<code>git add <file></code>
Modified → Staged	<code>git add <file></code>
Staged → Committed	<code>git commit -m "message"</code>
Committed → Modified	(edit the file)

6. .GITIGNORE EXAMPLE

Ignore unnecessary files and folders.

```

# Logs
*.log
npm-debug.log*
yarn-debug.log*

# Dependencies
dist/
node_modules/

# Environment
.env

# OS files
.DS_Store
Thumbs.db

# IDE
.vscode/
.idea/
  
```

Keep your repository clean and secure.



7. BEST PRACTICES

- ✓ Commit small, meaningful changes
- ✓ Write clear commit messages
- ✓ Pull before push to avoid conflicts
- ✓ Use branches for new features & bugs
- ✓ Keep main branch stable and protected
- ✓ Review code via Pull Requests
- ✓ Use .gitignore to exclude unwanted files
- ✓ Delete merged branches
- ✓ Tag releases (git tag -a v1.0.0 -m "release")
- ✓ Backup important repositories





**Narendra
Pratap Singh**
DevOps & Cloud Engineer

VERSION CONTROL

DEEP DIVE - PART 2

Master the remaining essentials of Git
and become a Version Control Expert!

WHAT WE COVERED SO FAR

- ✓ Core Concepts
- ✓ Git Architecture
- ✓ Basic & Most Used Commands
- ✓ Branching Strategies
- ✓ Best Practices



linkedin.com/in/narendra09-devops



github.com/narendra09-devops



DevOps Learning Hub
https://narendra09-devops.github.io/devops-learning-hub/



8. MERGE, REBASE & CONFLICTS

MERGE

Combines changes from one branch into another with a merge commit.

```
git checkout main
git merge feature/login
```

REBASE

Moves or reapplies commits on top of another base.

```
git checkout feature/login
git rebase main
```

CONFLICTS

Happen when Git can't automatically merge changes.

How to resolve:

1. Open conflicted files
2. Look for conflict markers

```
<<<< HEAD
your changes
|||||
incoming changes
>>>> dev branch
```

3. Edit and keep the correct changes
4. git add <file>
5. git commit

WHEN TO USE?



- Merge
 - Preserve history
 - Use for shared branches (main, develop)



- Rebase
 - Clean linear history
 - Use for feature branches



- Don't rebase public branches
- Always pull before you push
- Resolve conflicts calmly 😊

9. REMOTE REPOSITORIES & COLLABORATION



Add Remote
`git remote add origin <url>`



View Remotes
`git remote -v`



Fetch (Download)
`git fetch origin`



Pull (Fetch + Merge)
`git pull origin main`



Push (Upload)
`git push origin main`



Push New Branch
`git push -u origin feature/login`



Delete Remote Branch
`git push origin --delete feature/login`



Rename Remote
`git remote rename origin upstream`

REMOTE PROTOCOLS

HTTPS (Recommended)

Easy to use, secure.

SSH (Recommended)

More secure, key based auth.

COLLABORATION TIPS

- Communicate before big changes
- Pull latest before starting work
- Push small & frequent commits
- Use issues to track work

10. PULL REQUEST & CODE REVIEW



1. Create Branch
`git checkout -b feature/xyz`



2. Commit Changes
`git add .`
`git commit -m "Add xyz"`



3. Push Branch
`git push origin feature/xyz`



4. Open Pull Request
Compare & create PR on GitHub/GitLab



5. Code Review
Team reviews changes



6. Merge & Delete Branch
After approval, merge and delete branch

BENEFITS

- ✓ Catch bugs early
- ✓ Share knowledge
- ✓ Maintain code quality
- ✓ Audit trail of changes

BEST PRACTICES

- Write good PR descriptions
- Keep PR small and focused
- Respond to feedback
- Squash commits before merge

11. ADVANCED TOPICS



CHERRY-PICK

Apply a specific commit from one branch to another.

```
git cherry-pick <commit-hash>
```



STASH

Temporarily save uncommitted changes.

```
git stash
git stash pop
```



TAGGING

Mark specific points (releases) in history.

```
git tag -a v1.0.0 -m "Version 1.0.0"
git push origin v1.0.0
```



SUBMODULES

Include and manage external repositories.

```
git submodule add <repo-url> path/
git submodule update --init --recursive
```



GIT HOOKS

Run scripts on Git events.

Common Hooks: pre-commit, commit-msg, pre-push, post-merge



.gitattributes

Define how Git handles files.

```
*.txt text eol=lf
```

SQUASH COMMITS

Combine multiple commits into one.

```
git rebase -i HEAD~n
```

Change "pick" to "squash" and save. ✎

UNDO CHANGES

Unstage a file

```
git reset <file>
```

Discard changes in file

```
git checkout -- <file>
```

Undo last commit

```
git reset --soft HEAD~1
```

Undo last commit (discard changes)

```
git reset --hard HEAD~1
```

Revert a commit (safe for public)

```
git revert <commit-hash>
```

12. GIT ALIASES (Productivity Boost)

Alias	Command
co	checkout
br	branch
ci	commit
st	status
lg	log --oneline --graph --all
df	diff
dc	diff --cached
cm	commit -m
ps	push
pl	pull
gpo	push origin
gpsup	push -u origin
ga	git add .
gaa	git add --all
cl	clone
unstage	reset HEAD --
last	log -1 HEAD
type	cat-file -t
dump	cat-file -p

Add Alias

```
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.lg "log --oneline --graph --all"
```

View Aliases: `git config --global --get-regexp alias`

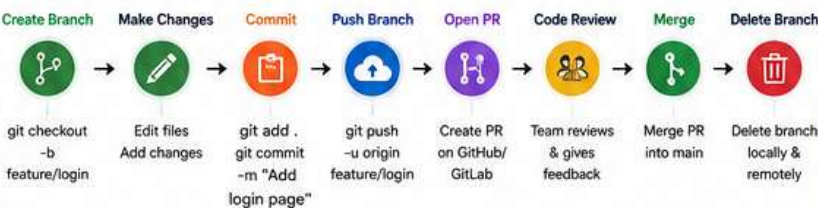
13. .GIT DIRECTORY - WHAT'S INSIDE?

The .git directory stores all metadata and history.	
HEAD	Points to current branch or commit
config	Repository configuration
objects/	All objects (commits, trees, blobs, tags)
refs/	References to branches, tags
refs/heads/	Local branches
refs/remotes/	Remote tracking branches
hooks/	Git hooks (scripts)
index	Staging area (also called cache)
logs/	Logs of all Git activities
info/	Global exclude file, commit templates
packed-refs	Compressed references

View .git Directory (safe way)

```
ls -la .git
```

14. WORKFLOW EXAMPLE (Feature to Production)



Golden Workflow Rules

- ★ Work on feature branches
- ★ Commit small and meaningful
- ★ Push & create Pull Request
- ★ Review, discuss and improve
- ★ Merge & ship with confidence

15. COMMON SCENARIOS & SOLUTIONS

Scenario	Solution
Accidentally committed sensitive data	<code>git filter-repo</code> or BFG Repo-Cleaner + force push (use with caution)
Committed to wrong branch	<code>git cherry-pick <commit></code>
Local branch behind remote	<code>git pull origin branch-name</code>
Local branch ahead of remote	<code>git push origin branch-name</code>
Untracked files blocking pull	<code>git stash</code> or <code>git clean -fd</code>
Need to go back to previous state	<code>git reset --hard <commit-hash></code>
Need to undo a pushed commit	<code>git revert <commit-hash></code>
Someone else's changes	Pull, resolve conflicts, commit, push conflicting



Great DevOps Engineers don't just write code,
they manage changes like Pros!
Version everything. Trust every deployment.



REMEMBER

A good version control practice today
saves hours of pain tomorrow.

KEEP LEARNING. KEEP BUILDING.
LET'S GROW TOGETHER!



linkedin.com/in/narendra09-devops



github.com/narendra09-devops



DevOps Learning Hub

https://narendra09-devops.github.io/devops-learning-hub/





**Narendra
Pratap Singh**
DevOps & Cloud Engineer

VERSION CONTROL DEEP DIVE – PART 3

Advanced Git Topics, Tools, and Best Practices

★ You've learned the essentials. Now level up with advanced Git skills and tools!

WHAT WE COVERED SO FAR

- ✓ Core Concepts & Architecture
- ✓ Basic & Most Used Commands
- ✓ Branching Strategies
- ✓ Workflows & Collaboration
- ✓ Advanced Topics (Part 1 & 2)



linkedin.com/in/narendra09-devops



github.com/narendra09-devops



DevOps Learning Hub
https://narendra09-devops.github.io/devops-learning-hub/



16. STASH – SAVE WORK IN PROGRESS

Stash lets you save your uncommitted changes and clean your working directory.

Common Stash Commands

<code>git stash</code>	Stash current changes
<code>git stash list</code>	List all stashes
<code>git stash push -m "msg"</code>	Stash with message
<code>git stash pop</code>	Apply latest stash & drop it
<code>git stash apply</code>	Apply latest stash (keep it)
<code>git stash drop</code>	Drop latest stash
<code>git stash clear</code>	Remove all stashes

💡 Use stash to quickly switch context without committing half-done work.

17. REWRITING HISTORY (USE CAREFULLY)

AMEND LAST COMMIT

```
git commit --amend -m "new message"
```

Modify the most recent commit.

INTERACTIVE REBASE

```
git rebase -i HEAD~n
```

Edit, reorder, squash or remove last n commits.

COMMON ACTIONS IN EDITOR

pick	Use commit
reword	Use commit but edit message
edit	Stop to amend commit
squash	Merge with previous commit
fixup	Like squash but discard message
drop	Remove commit

⚠️ Never rewrite history on shared branches!

18. CHERRY-PICK

Apply a specific commit from one branch to another.



```
git cherry-pick <commit-hash>
```

💡 Useful for hotfixes, bug fixes, or picking a single commit without merging the whole branch.

19. SUBMODULES & SUBTREES

SUBMODULES

- Include external repositories as a submodule.
- Keeps history separate.
- Good for large dependencies.

```
git submodule add <repo-url> path/
```

SUBTREES

- Bring another repo's code into your repo history.
- Simpler than submodules.
- Good for smaller dependencies.

```
git subtree add --prefix path <repo-url> <branch>
```

20. GIT HOOKS

Hooks are scripts that Git runs automatically on certain events.

Hook (Event)	When it runs	Use Cases
pre-commit	Before a commit is created	Run linters, tests
commit-msg	Before commit message is saved	Validate commit message format
pre-push	Before pushing to remote	Run tests, scans
post-receive	After push to remote (server-side)	Deploy, notify, logging
pre-receive	Before push is accepted (server-side)	Reject bad commits

Example: pre-commit hook (run tests)

```
#!/bin/sh
echo "Running tests before commit..."
# run your tests
echo "All tests passed 🟢"
exit 0
```

💡 Hooks help enforce quality and automation in your workflow.

21. .GIT DIRECTORY – UNDER THE HOOD

Key files & folders inside .git

HEAD	Points to current branch/commit
config	Repository configuration
objects/	All Git objects (blobs, trees, commits, tags)
refs/	References (branches, tags, remotes)
logs/	Logs for relog and branch updates
hooks/	Git hooks scripts
index	Staging area (index file)
packed-refs	Optimized references

💡 Understanding .git helps in advanced troubleshooting and recovery.

22. TAGS – MARK IMPORTANT POINTS

Types of Tags

Lightweight Tag

Simple pointer to a commit.

```
git tag v1.0.0
```

Annotated Tag

Stores extra metadata.

```
git tag -a v1.0.0 -s "Version 1.0.0"
```

Common Tag Commands

<code>git tag</code>	List all tags
<code>git tag -l "v1.*"</code>	List tags matching pattern
<code>git show v1.0.0</code>	Show tag details
<code>git push origin v1.0.0</code>	Push tag to remote
<code>git push origin --tags</code>	Push all tags

★ Use tags for releases, milestones and versioning.

23. GIT IN TEAM ENVIRONMENTS

- ✓ Always pull latest changes before starting work
- ✓ Use feature branches for new work
- ✓ Create Pull Requests for review & discussion
- ✓ Write clear commit messages
- ✓ Keep commits atomic and focused
- ✓ Respect branch protection rules
- ✓ Communicate in PRs and code reviews
- ✓ Delete merged branches
- ✓ Keep main branch stable and deployable

👥 Good process + good tools = Great teams!

24. SECURITY & BEST PRACTICES

- ✓ Use SSH keys, not passwords
- ✓ Enable branch protection (require PR, reviews)
- ✓ Scan for secrets (e.g., Gitleaks)
- ✓ Don't commit secrets or credentials
- ✓ Use .gitignore properly
- ✓ Keep dependencies updated
- ✓ Backup important repositories
- ✓ Audit your repository activity

🛡️ Secure code. Secure pipeline. Secure everything.

25. TROUBLESHOOTING CHEAT SHEET

Problem	Solution / Command
Accidentally committed wrong changes	<code>git reset --soft HEAD~1</code>
Need to discard local changes	<code>git restore .</code>
Need to discard a file	<code>git restore <file></code>
Pushed secrets / sensitive data	Use BFG Repo-Cleaner or <code>git filter-repo</code>
Lost commits	<code>git reflog</code>
Detached HEAD state	<code>git switch -c new-branch</code>
Merge conflicts	Resolve manually, then <code>git add</code> & <code>git commit</code>
Want to see file history	<code>git log --follow <file></code>
See who changed what	<code>git blame <file></code>

💡 When in doubt, check `git log`, `git status` and `git reflog`!

26. USEFUL TOOLS & VISUAL CLIENTS

CLI Tools

- Git CLI**
The official command line tool.
- GitHub CLI (gh)**
Work with GitHub from terminal.
- GitLab CLI**
Interact with GitLab projects.
- LazyGit**
Terminal UI for Git (awesome!).

GUI Tools

- VS Code (Built-in)**
Great Git integration.
- SourceTree**
Powerful Git GUI.
- GitKraken**
Beautiful Git client.
- TortoiseGit (Windows)**
Windows Explorer integration.

27. NEXT STEPS – LEVEL UP

- 🔗 Learn Git Internals (objects, plumbing commands)
- 🔗 Practice advanced workflows
- 🔗 Explore GitHub Actions for automation
- 🔗 Implement GitOps with Git
- 🔗 Contribute to open source projects
- 🔗 Build and maintain your own projects



GIT IS A SUPERPOWER.

PRACTICE DAILY. BUILD CONFIDENCE. SOLVE BIG PROBLEMS.

Keep Learning. Keep Building.

Let's Grow Together! 🚀



linkedin.com/in/narendra09-devops



github.com/narendra09-devops



DevOps Learning Hub
https://narendra09-devops.github.io/devops-learning-hub/





Narendra
Pratap Singh
DevOps & Cloud Engineer

VERSION CONTROL

DEEP DIVE - PART 4

Advanced Concepts, Collaboration Models,
and Real-World Best Practices

WHAT WE COVERED SO FAR

- ✓ Core Concepts & Architecture
- ✓ Basic & Most Used Commands
- ✓ Branching Strategies
- ✓ Workflows & Collaboration
- ✓ Advanced Topics & Tools



linkedin.com/in/narendra09-devops



github.com/narendra09-devops



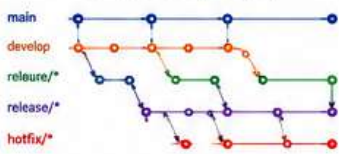
DevOps Learning Hub

https://narendra09-devops.github.io/devops-learning-hub/



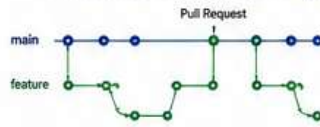
28. ADVANCED WORKFLOWS & MODELS

Git Flow (For Large Projects)



- Structured workflow with long-lived branches
- Best for products with scheduled releases
- Tools: git-flow, GitKraken

GitHub Flow (For Continuous Delivery)



- Create branch → Commit → Open PR
- Review → Merge → Deploy
- Best for CI/CD and small teams

Trunk Based Development



- Short-lived feature branches
- Frequent commits to main
- Requires strong CI, automated tests

Forking Workflow (Open Source)

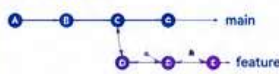


- Fork → Create branch → Commit → PR
- Sync with upstream regularly
- Common in open source projects

29. GIT REBASE DEEP DIVE

Rewriting history by moving or combining a sequence of commits.

BEFORE REBASE



AFTER REBASE



WHEN TO USE?

- ✓ Keep history clean and linear
- ✓ Before creating Pull Request
- ✓ To apply changes on top of latest base branch

CAUTION

- Never rebase shared branches
- May cause conflicts
- Coordination is key

30. INTERACTIVE REBASE

Edit, reorder, squash, or drop commits.

```
git rebase -i HEAD~5
```

pick	Use commit
reword	Edit commit message
edit	Modify commit
squash	Merge with previous
fixup	Merge without keeping message
drop	Remove commit

EXAMPLE USE CASES

- ✓ Combine multiple WIP commits
- ✓ Fix commit messages
- ✓ Remove unnecessary commits
- ✓ Reorder commits for clarity

31. GIT DIFF EXPLAINED

Compare Changes

git diff	# Working directory vs Staging
git diff --staged	# Staging vs Last commit
git diff HEAD	# Working directory vs Last commit
git diff branch1..branch2	# Compare two branches
git diff --stat	# Summary of changes

29. GIT LOG - POWERFUL OPTIONS

Command	Description
git log	Show commit history
git log --oneline	Compact one-line output
git log --graph --oneline --all	Visual graph of all branches
git log --author="name"	Filter by author
git log --since="2024-01-01"	Commits after date
git log --until="2024-12-31"	Commits before date
git log -S"keyword"	Search commits by changes
git log -p	Show full patch of changes
git blame <file>	Show last change per line

Tip: Use 'q' to quit log pages, '/' to search in logs.

33. RECOVERING & UNDOING THINGS

Situation	Command
Undo last commit (keep changes)	git reset --soft HEAD~1
Undo last commit (remove changes)	git reset --hard HEAD~1
Undo last commit (safe)	git revert HEAD
Unstage a file	git reset <file>
Discard changes in file	git restore <file>
Discard all local changes	git reset --hard
Recover deleted branch	git reflog git checkout -b <branch-name> <commit-id>

Be careful with --hard. It deletes changes permanently.

34. PERFORMANCE & MAINTENANCE

Clean untracked files	git clean -fd
Remove untracked files (preview)	git clean -fdn
Optimize local repo	git gc
Verify repository	git fsck
Show repo size	du -sh .git
Count objects	git count-objects -vH
Repack repository	git repack -a -d

Regular maintenance keeps your repo fast and healthy.

35. GIT CONFIGURATION

Scope	Description	Example
System	For all users on system	git config --system user.name "John"
Global	For current user	git config --global user.email "you@example.com"
Local	For current repository	git config user.name "Repo Name"

Useful Configs

- Set default branch name: `git config --global init.defaultBranch main`
- Enable color output: `git config --global color.ui auto`
- Set merge tool: `git config --global merge.tool vimdiff`
- Show config: `git config --list --show-origin`
- Set editor: `git config --global core.editor "code --wait"`

36. LARGE FILES & LFS

THE PROBLEM

Git is not optimized for large binary files (>100MB). Repo size grows fast.

SOLUTION: GIT LFS

- ✓ Store large files in Git LFS
- ✓ Keep pointers in Git repo
- ✓ Improves performance

Common Commands

```
git lfs install
git lfs track "*.psd"
git add .gitattributes
git add large-file.psd
git commit -m "Add large file"
git push
```

37. SUBMODULES VS SUBTREES



SUBMODULES

Reference another repo inside your repo.

- Keep isolated history
- Need to init & update
- Good for independent projects

```
git submodule add <repo-url> path/
```



SUBTREES

Bring another repo's code into your repo.

- Simple to use
- Single repo history
- Good for smaller dependencies

```
git subtree add --prefix=path <branch>
```

38. CODE REVIEW & QUALITY

- ✓ Small PRs are easier to review
- ✓ Write clear commit messages
- ✓ Follow commit message convention
- ✓ Automate checks (Lint, Test, Build)
- ✓ Use Pull Request templates
- ✓ Review for logic, security, performance
- ✓ Use Code Owners for critical paths

Commit Message Convention (Example)

```
feat(auth): add login with OAuth
Fixes #123
Co-authored-by: John Doe <john@example.com>
```

39. COMMIT MESSAGE BEST PRACTICES

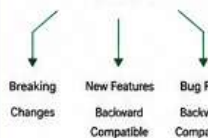
feat	A new feature	feat: add user registration
fix	A bug fix	fix: resolve login issue
docs	Documentation only	docs: update readme
style	Code style changes	style: format code with prettier
refactor	Code refactoring	refactor: improve error handling
test	Adding or updating tests	test: add unit tests for auth
chore	Maintenance tasks	chore: update dependencies

40. VERSIONING & TAGGING

SEMANTIC VERSIONING

MAJOR.MINOR.PATCH

1.2.3



GIT TAGS

```
git tag # List tags
git tag -a v1.0.0 -m "Version 1.0.0" # Create annotated tag
git push origin v1.0.0 # Push single tag
git push origin --tags # Push all tags
```

Why Use Tags?

- Mark releases
- Deploy specific versions
- Rollback easily
- Trace history

41. WHAT'S NEXT?

- ✦ Learn Git Internals (objects, plumbing commands)
- ✦ Explore advanced tooling & automation
- ✦ Implement branch protection & policies
- ✦ Integrate with CI/CD pipelines
- ✦ Contribute to open source
- ✦ Keep practicing with real projects

Great Version Control =
Better Collaboration + Safer Releases
+ Happier Teams! 🚀



PRACTICE DAILY. BUILD CONFIDENCE.
SOLVE BIG PROBLEMS.

KEEP LEARNING. KEEP BUILDING.
LET'S GROW TOGETHER! 🚀



linkedin.com/in/narendra09-devops



github.com/narendra09-devops



DevOps Learning Hub

https://narendra09-devops.github.io/devops-learning-hub/





42. BRANCH PROTECTION (BEST PRACTICE)

Protect important branches (main, release/*)

Common Rules

- Require Pull Request before merging
- Require Approvals (1 or more reviewers)
- Require Status Checks (CI/CD must pass)
- Require Up-to-date Branch
- Limit who can push / force push
- Allow force push (Never for main ✗)
- Delete branch after merge (optional)

Why? Keeps code quality high & production safe

43. CI/CD INTEGRATION WITH GIT



Best Practices

- Build on every PR and Push
- Automatic Deploy after Merge
- Run Tests & Quality Checks
- Use Environments & Approvals
- Block Merge if Pipeline Fails
- Notify on Success/Failure

44. GIT IN LARGE TEAMS (BEST PRACTICES)

- Follow a workflow (Git Flow / Trunk Based)
- Clear Ownership of branches
- Small & frequent commits
- Write meaningful commit messages
- Code review is mandatory
- Communicate before big changes
- Keep documentation with code (README.md)

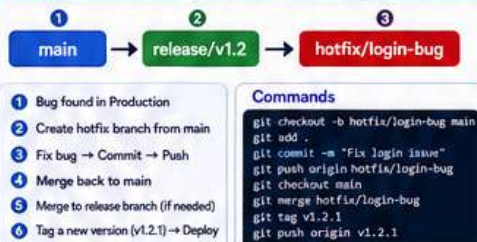
PROCESS + COMMUNICATION = SUCCESS

45. MONOREPO vs MULTI-REPO



Choose based on Team Size, Project Complexity & Tooling

46. REAL-WORLD SCENARIO - HOTFIX TO PRODUCTION



Always test hotfix in lower env before production deploy!

47. HANDLING VERSIONS USING TAGS

Create & Push Tags

```
git tag v2.0.0
git tag -a v2.0.0 -m "Release 2.0.0"
git push origin v2.0.0
git push origin --tags
```

View & Manage Tags

```
git tag # list tags
git show v2.0.0 # view tag details
git tag -d v2.0.0 # delete local tag
git push origin --delete v2.0.0 # delete remote tag
```

Semantic Versioning (MAJOR.MINOR.PATCH)

2.0.0 Breaking Change (Incompatible) | 2.1.0 New Feature (Backward Compatible) | 2.1.1 Bug Fix (Backward Compatible)

48. GIT WORKFLOW CHEAT SHEET (SUMMARY)

Task	Commands
Start New Repo	git init
Clone Repo	git clone <url>
Create Branch	git checkout -b feature/xyz
Switch Branch	git checkout branch-name
Add Files	git add .
Commit	git commit -m "message"
Push	git push origin branch-name
Pull Latest	git pull origin branch-name
Merge Branch	git merge branch-name
Rebase Branch	git rebase branch-name
Tag Release	git tag v1.0.0 && git push --tags
Stash	git stash && git stash pop
Undo Commit (keep changes)	git reset --soft HEAD~1
Undo Commit (discard)	git reset --hard HEAD~1

Bookmark this! You'll use this daily.

49. .GITIGNORE - SMART EXAMPLES

Common .gitignore Template (Web/General)

```
# OS Files
.DS_Store
Thumbs.db

# Logs
logs/
*.log
npm-debug.log*

# Node / Python / Java
node_modules/
venv/
__pycache__/
*.class
*.jar

# Environment
.env
.env.*

# IDE
.vscode/
.idea/
*.swp

# Build / Dist
build/
dist/
target/

# Others
coverage/
*.tmp
*.cache
```

Why Ignore?

- Reduce Repo Size
- Avoid Sensitive Data Leak
- Faster Clones & Pulls
- Clean History
- Rule: Ignore what can be generated, local or sensitive.

50. ENTERPRISE & TEAM POLICIES

Commit Message Policy
Follow conventional commits

Branch Naming Policy
feature/jira-123, bugfix/jira-456

PR Policy
Review + Approval Mandatory

Code Quality
Lint, Unit Test, Security Scan

Release Policy
Tag, Changelog, Release Notes

Backup & Recovery
Regular backups of repos

Access Control
Role based access (Read/Write/Admin)

Conventional Commits

- feat: new feature
- fix: bug fix
- docs: documentation
- style: code style
- refactor: code refactor
- test: add tests
- chore: maintenance

Example:
feat(auth): add login with OAuth
fix(ui): button alignment issue
chore: update dependencies

51. DISASTER RECOVERY & BACKUP

Pask	Recovery Options	Useful Commands
Accidental Delete	Use git reflog	git reflog # find lost commit
Data Corruption	Restore from Backup	git reset --hard <commit-id>
Force Push Gone Wrong	Use Remote Repo	git fetch --lost-found
Server Outage	Use CI/CD Artifacts	git show <commit-id>
		git restore --source=<commit-id>

! You'll take egnorly and don't force push to main!

52. REAL-TIME TROUBLESHOOTING

Problem	Solution
Accidentally deleted branch	git reflog -> find commit -> git checkout -b new-branch -id
Already pushed wrong commit	git revert <commit-id> (safe) OR reset (if not shared)
Merge conflict	Resolve files -> git add -> git commit
Detached HEAD	git checkout -b new-branch
Stash lost	git stash list -> git stash apply stash@{n}
Large file pushed	Use Git LFS, git filter-repo (remove)

53. WHAT'S NEXT AFTER GIT?

Up Your Game!

- Git Internals (Objects, DAG)
- Git Plumbing Commands
- Git Hooks Advanced Usage
- GitOps (ArgoCD, Flux)

- Policy as Code (Open Policy Agent)
- Advanced CI/CD Pipelines
- Security Scanning (SAST, Secrets)
- Open Source Contributions

Never Stop Learning. Keep Building!

Great DevOps Engineers
Own Their Code from
COMMIT to CLOUD!

CODE. COMMIT. COLLABORATE.
DEPLOY. DELIVER. REPEAT.



Scan to Visit
DevOps Learning Hub





**Narendra
Prapat Singh**
DevOps & Cloud Engineer

VERSION CONTROL

DEEP DIVE – PART 5 (FINAL)

Advanced Collaboration, Real-World Practices
and Everything You Must Know!

WHAT WE COVERED SO FAR

- ✓ Core Concepts & Architecture
- ✓ Basic & Most Used Commands
- ✓ Branching Strategies
- ✓ Workflows & Collaboration
- ✓ Advanced Topics & Tools

[linkedin.com/in/narendra09-devops](https://www.linkedin.com/in/narendra09-devops)

github.com/narendra09-devops

DevOps Learning Hub
<https://narendra09-devops.github.io/devops-learning-hub/>



★ You now have everything to become a Git Expert. Practice. Build. Share. Grow!

42. ADVANCED COLLABORATION PATTERNS



PAIR PROGRAMMING FLOW

- 1 Create branch
- 2 Pair works & commits
- 3 Push branch
- 4 Create PR
- 5 Review & Merge

Small commits, frequent pushes, constant sync.



CODE REVIEW CHECKLIST

- ✓ Functionality
- ✓ Code readability
- ✓ Performance
- ✓ Security
- ✓ Tests added/updated
- ✓ Documentation
- ✓ Edge cases

Review smart, not just fast.



EFFECTIVE PR TITLES

- ✓ feat: add user registration
- ✓ fix: resolve login issue
- ✓ refactor: improve auth module
- ✓ docs: update README
- ✓ test: add unit tests for payment

Use conventional commit style for clarity.

43. PROTECTED BRANCHES



Protect important branches like main, develop, release/*

- ✓ Require Pull Request
- ✓ Require approvals
- ✓ Require status checks (CI/CD)
- ✓ Dismiss stale reviews
- ✓ Restrict who can push

Protection ensures code quality and prevents accidental changes.

44. .GITIGNORE DEEP DIVE

Common entries

```
.DS_Store          # macOS files
node_modules/      # Node dependencies
*.log              # Log files
.env              # Environment files
__pycache__/*      # Python cache
*.pyc              # Python compiled
dist/              # Distribution folder
build/             # Build output
*.idea/            # IDE files (IntelliJ)
.vscode/           # VS Code settings
*.zip              # Archives
```

Best Practices

- ✓ Keep it in repo root
- ✓ Ignore secrets & keys
- ✓ Ignore build outputs
- ✓ Ignore local configs
- ✓ Review before commit

A clean repo is a professional repo!

45. GIT CONFIGURATION BEST PRACTICES

Essential Global Configs

```
git config --global user.name "Your Name"
git config --global user.email "you@email.com"
git config --global core.editor "code --wait"
git config --global init.defaultBranch main
git config --global pull.rebase false
git config --global core.autocrlf input
```

Useful Configs

```
✓ color.ui auto      # Better output
✓ alias.co checkout  # Shortcuts
✓ alias.br branch    # Shortcuts
✓ alias.ci commit    # Shortcuts
✓ alias.st status     # Shortcuts
✓ pull.rebase false  # Safer for beginners
✓ core.autocrlf input # Cross-platform
```

Run `git config --list` to see all configurations.

46. GIT ALIASES YOU SHOULD USE

```
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.ci commit
git config --global alias.unstage 'reset HEAD--'
git config --global alias.last 'log -1 HEAD'
git config --global alias.visual '!gitk'
git config --global alias.graph 'log --oneline --graph --all'
```

Aliases save time. Time saved = More productivity!

47. COMMIT CONVENTIONS (CONVENTIONAL COMMITS)

Use conventional commits for automated changelogs & releases.

```
feat      A new feature
fix       A bug fix
docs      Documentation only
style     Code style (formatting, missing semi)
refactor  Refactoring code
test      Adding or updating tests
chore     Maintenance tasks
perf      Performance improvements
ci        CI/CD changes
build     Build system or dependencies
```

Example: `feat(auth): add login with OAuth`

48. CI/CD WITH GIT – BEST PRACTICES

- Trigger builds on PR and push
- Run tests before merge
- Block merge on failed checks
- Deploy only from protected branches
- Use semantic versioning for releases
- Tag releases and create changelogs
- Use Git tags to trigger production deploys

FLOW EXAMPLE



49. REAL-WORLD REPOSITORY STRUCTURE EXAMPLE

```
my-project/
├── .github/           # GitHub Actions, templates
├── docs/              # Documentation
├── src/               # Source code
├── tests/             # Tests
├── scripts/           # Helper scripts
├── .gitignore         # Ignore rules
├── README.md          # Project overview
├── CHANGELLOG.md      # All notable changes
├── LICENSE            # License file
└── CONTRIBUTING.md    # Contribution guide
```

A well-structured repo is easy to maintain and scale.

50. HANDY ONE-LINERS

```
Undo last commit (keep changes)  git reset --soft HEAD~1
Undo last commit (discard changes) git reset --hard HEAD~1
Undo last pushed commit          git revert HEAD
Show files changed in a commit    git show --name-only <commit>
Show files changed between branches git diff main..feature
Compare two commits               git diff <c1> <c2>
List branches merged to main      git branch --merged main
List branches not merged to main  git branch --no-merged main
Find who changed a line           git blame <file>
See commit history graph          git log --oneline --graph --all
Clean untracked files & folders    git clean -fd
```

51. COMMON GIT SCENARIOS – SOLUTIONS

Scenario	Solution
Committed to wrong branch	<code>git cherry-pick <commit></code>
Need to move last commit to another branch	<code>git checkout new-branch</code> <code>git cherry-pick <commit></code>
Need to undo local changes	<code>git restore <file></code>
Need to undo all local changes	<code>git reset --hard</code>
Need to undo pushed commit	<code>git revert <commit></code>
Need to rename a branch	<code>git branch -m old new</code>
Need to delete a branch locally	<code>git branch -d <branch></code>
Need to delete a branch remotely	<code>git push origin --delete <branch></code>
Need to sync fork	<code>git fetch upstream</code> <code>git checkout main</code> <code>git merge upstream/main</code> <code>git push origin main</code>

Every problem has a Git solution. Learn the patterns!

52. SECURITY & COMPLIANCE TIPS

- ✓ Never commit secrets (API keys, passwords)
- ✓ Use environment variables
- ✓ Use .gitignore for sensitive files
- ✓ Scan for secrets (e.g., Gitleaks, TruffleHog)
- ✓ Use signed commits (GPG)
- ✓ Audit repository activity regularly
- ✓ Keep dependencies updated
- ✓ Use branch protection & required reviews
- ✓ Follow least privilege access

Security in Git is Security in DevOps.

53. PRACTICE PROJECT IDEAS

- Build a personal portfolio website & host on GitHub Pages
- Create a DevOps Learning Hub (You're doing great!)
- Contribute to open source
- Build a CLI tool and version it
- Practice Git workflows with teams
- Automate CI/CD for a sample app
- Maintain a changelog and release versions

Practice daily. Build projects. Share knowledge.

54. FINAL CHECKLIST

- ✓ Understand Git concepts deeply
- ✓ Use branches effectively
- ✓ Write good commit messages
- ✓ Collaborate using PRs
- ✓ Protect important branches
- ✓ Resolve conflicts confidently
- ✓ Follow best practices
- ✓ Keep learning & building



**YOU NOW HAVE THE COMPLETE
KNOWLEDGE OF VERSION CONTROL!**

Keep Learning. Keep Building.

Let's Grow Together!



[linkedin.com/in/narendra09-devops](https://www.linkedin.com/in/narendra09-devops)



github.com/narendra09-devops



DevOps Learning Hub
<https://narendra09-devops.github.io/devops-learning-hub/>



Scan me!



**GIT IS POWER.
USE IT WISELY.**

**GREAT CODE STARTS WITH
GREAT VERSION CONTROL.**

**BETTER COLLABORATION • SAFER RELEASES
HAPPIER TEAMS**

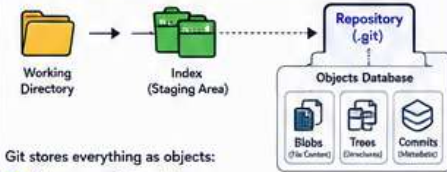


- ✓ Core Concepts & Architecture
- ✓ Commands (Basic to Advanced)
- ✓ Branching Strategies & Workflows
- ✓ Collaboration & Tools
- ✓ Advanced Topics (Rebase, Hooks, etc.)
- ✓ Enterprise Practices & Real-World Scenarios



54. GIT INTERNALS - WHAT HAPPENS UNDER THE HOOD?

Git is not just a list of files; it's a content-addressable filesystem.



Git stores everything as objects:

- Blobs → File contents
- Trees → Directory structure
- Commits → Snapshot + metadata + parent link

Key Insight: Every commit points to a tree, which points to blobs.

That's how Git is fast, scalable & reliable!

55. GIT OBJECTS EXPLAINED

Object	Description	Command to View
Blob	Store file contents (binary/text)	git cat-file -p <blob-hash>
Tree	Store directory structure and file names	git cat-file -p <tree-hash>
Commit	Store snapshot + author, committer, parent	git cat-file -p <commit-hash>
Tag	Store references to commits (with metadata)	git cat-file -p <tag-hash>

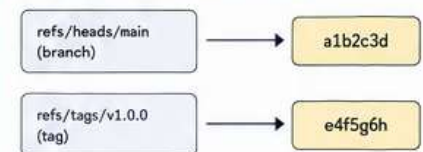
Example:

```
git rev-parse HEAD # Get commit hash
git cat-file -p <commit-hash> # See commit details
git ls-tree -r <commit-hash> # List files in a commit
```

★ Understanding Git objects = Understanding Git Power!

56. GIT REFS - POINTERS IN GIT

Git uses references (refs) to point to commits.



View all refs:

```
git show-ref
```

Refs are just pointers. When you move a branch, you move the pointer, not the commits!

57. GIT HOOKS DEEPR LOOK

Hooks run at important events.

Hook	When it runs	Use Cases
pre-commit	Before commit	Lint, format, run tests
commit-msg	Before commit message is recorded	Validate commit message
pre-push	Before pushing	Run tests, security scan
post-merge	After merge	Notify, run build
pre-receive	On server, before push is accepted	Enforce policies
update	On server, after refs are updated	Logging, notifications

Best Practice: Use hooks to automate quality, security & consistency!

58. GIT SUBMODULES VS SUBTREES VS MONOREPO

Submodules	Subtrees	Monorepo
Repo inside a repo, linked to external repo.	Bring another repo's code into your repo.	All projects in a single repository.
Pros • Keep repos separate • Reuse across projects	Pros • Simple to use • No extra repo	Pros • Easy dependency mgmt • Atomic commits
Cons • Complex to manage • Detached HEAD issues	Cons • History gets mixed	Cons • Repo can grow large

Choose wisely based on your:
Team Size | Project Complexity | Tooling | Deployment Model

59. LARGE BINARY FILES - GIT LFS

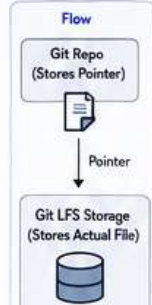
Problem: Git is not optimized for large files (images, videos, datasets, ML models).

Solution: Git LFS (Large File Storage)

- ✓ Stores large files outside Git repo
- ✓ Keeps pointers in Git
- ✓ Improves performance

Common Commands

```
git lfs install # Install LFS
git lfs track "*.psd" # Track file types
git add .gitattributes # Add attributes file
git add large-file.psd # Add large file
git commit -m "Add large file with LFS"
git push
```



Great for: Media, Datasets, ML Models, CAD files etc.

60. GIT IGNORE BEST PRACTICES

Example .gitignore

```
# OS
.DS_Store
Thumbs.db

# IDE
.vscode/
.idea/
*.suo
*.log
*.nps-debug.log

# Environment
.env
*.env.*

# Dependencies
node_modules/
vsn/
__pycache__
*.jar

# Build
build/
dist/
*.jless

# Others
*.tmp
coverage/
```

Tips

- ✓ Keep .gitignore in root
- ✓ Ignore local env files
- ✓ Commit .gitignore to repo
- ✓ Use templates (GitHub .gitignore templates)

Check ignored files

```
git status --ignored
```

⚠ Never commit secrets or credentials!

61. REAL-WORLD SCENARIO - HOTFIX WORKFLOW



- 1 Bug found in Production (v1.2)
- 2 Create hotfix branch from main
- 3 Fix bug → Commit → Push
- 4 Merge to release/1.2 (if needed)
- 5 Merge to main
- 6 Create tag v1.2.1
- 7 Deploy

```
Commands
git checkout main
git pull
git checkout -b hotfix/login-bug
git commit -m "Fix login issue"
git push origin hotfix/login-bug
git checkout release/1.2
git merge hotfix/login-bug
git checkout main
git merge hotfix/login-bug
git tag v1.2.1
git push origin main --tags
```

Best Practices

- ✓ Keep hotfixes small & focused
- ✓ Test before merging
- ✓ Tag every production release
- ✓ Update CHANGELOG.md
- ✓ Communicate with team

62. COMPLIANCE & AUDIT WITH GIT

Git helps in audits & compliance.

- 🕒 Complete history of changes
- 👤 Who changed what & when
- 🔄 Easy to rollback
- 🔒 Access control & branch protection
- ✅ Signed commits & verified authors

Enable Signed Commits

```
git config --global user.signingkey <key-id>
git config --global commit.gpgsign true
```

Verify Commit

```
git log --show-signature
```

Great for: ISO, SOC2, GDPR, HIPAA, Enterprise Audits

63. GIT CONFIGURATION ESSENTIALS

View Config

```
git config --list --show-origin
```

Set Config

Level	Command	Use Case
System	git config --system <key> <value>	All users on system
Global	git config --global <key> <value>	Your user account
Local (Repo)	git config --local <key> <value>	Specific repository

Common Configs

```
git config --global user.name "Narendra Pratap Singh"
git config --global user.email "narendra09@example.com"
git config --global core.editor "code --wait"
git config --global pull.rebase false
git config --global init.defaultBranch main
git config --global core.autocrlf input
```

64. USEFUL ALIASES (SAVE TIME)

Alias	Command	Use
co	checkout	Switch branches
br	branch	List branches
cl	commit	Commit changes
st	status	Repo status
lg	log --oneline --graph --all	Pretty log
df	diff	File changes
cm	commit -m	Commit with message
ps	push	Push changes
pl	pull	Pull changes
po	push origin	Push to origin
ppo	push origin --tags	Push tags
unstage	reset HEAD --	Unstage files
last	log -1 HEAD	Last commit
type	cat-file -t	Show object type

Add Aliases

```
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.lg "log --oneline --graph --all"
```

65. PRO TIPS FROM EXPERIENCE

- 📝 Write meaningful commit messages
- 📦 Commit small & atomic changes
- 🔄 Pull before you push
- 🔍 Resolve conflicts locally & test
- 🚫 Don't commit to main directly
- 🌿 Use branches for features & fixes
- 🏷️ Tag releases & maintain changelog
- 🤖 Automate with CI/CD
- 🧹 Keep repos clean & lightweight
- 💾 Backup important repos

Commit Message Format

```
<type>(<scope>): <subject>
<body>
<footer>
```

Example

```
feat(auth): add OAuth2 login
Implemented Google OAuth2 login flow for users.
```

Closes #123

Types: feat, fix, docs, style, refactor, test, chore, perf



REMEMBER

- ✓ Version Control is the foundation of DevOps.
- ✓ Master Git = Master Collaboration, Automation & Delivery!

KEEP LEARNING. KEEP BUILDING.
LET'S GROW TOGETHER!



linkedin.com/in/narendra09-devops

github.com/narendra09-devops

DevOps Learning Hub

https://narendra09-devops.github.io/devops-learning-hub/



Practice Daily. Build Confidence. Solve Real Problems.

You've got this! Keep pushing forward! 🚀



**Narendra
Pratap Singh**

DevOps & Cloud Engineer

VERSION CONTROL DEEP DIVE – PART 1

Git Foundation for DevOps Engineers

Master the basics before moving to branching, release, CI/CD and production workflows.



linkedin.com/in/narendra09-devops



github.com/narendra09-devops



DevOps Learning Hub

https://narendra09-devops.

github.io/devops-learning-hub/



Scan to Visit
DevOps Learning Hub

1 WHAT IS VERSION CONTROL?

Version Control helps track every change in code, scripts, configs, infrastructure files and documentation.

For DevOps Engineer, it is used for:

- Application code
- Shell/Python scripts
- Terraform files
- Ansible playbooks
- Kubernetes manifests
- Helm charts
- CI/CD pipeline files
- Documentation



2 WHY DEVOPS ENGINEERS NEED GIT

Git helps DevOps teams to:

- ✓ Track who changed what
- ✓ Rollback wrong changes
- ✓ Collaborate with team
- ✓ Maintain audit history
- ✓ Review changes before merge
- ✓ Automate CI/CD pipelines
- ✓ Version infrastructure as code



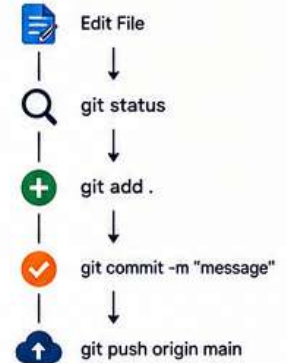
3 GIT ARCHITECTURE

Main Git areas:



- **Working Directory**
Where you edit files.
- **Staging Area**
Where you prepare files before commit.
- **Local Repository**
Your local Git history.
- **Remote Repository**
GitHub / GitLab / Bitbucket hosted repo.

4 BASIC GIT WORKFLOW



5 MOST USED GIT COMMANDS

```
git init
git clone <repo-url>
git status
git add .
git commit -m "message"
git log
git diff
git push
git pull
git fetch
```

6 GIT CONFIG COMMANDS

```
git config --global user.name "Narendra Pratap Singh"
git config --global user.email "your-email@example.com"
git config --list
```

7 REPOSITORY COMMANDS

```
git init
git clone <repo-url>
git remote -v
```



Remote -v shows the remote connections for your repository.

8 DAILY WORKFLOW COMMANDS

```
git status
git add .
git commit -m "Updated DevOps notes"
git push origin main
git pull origin main
```

9 FILE STATE IN GIT



- **Untracked** : New file not added to Git
- **Modified** : Existing file changed
- **Staged** : Ready for commit
- **Committed** : Saved in Git history

10 BEST PRACTICES

- ✓ Commit small changes
- ✓ Write clear commit messages
- ✓ Pull before push
- ✓ Never commit passwords
- ✓ Use .gitignore
- ✓ Keep main branch stable
- ✓ Use branches for new work
- ✓ Review changes before commit

11 DEVOPS EXAMPLE

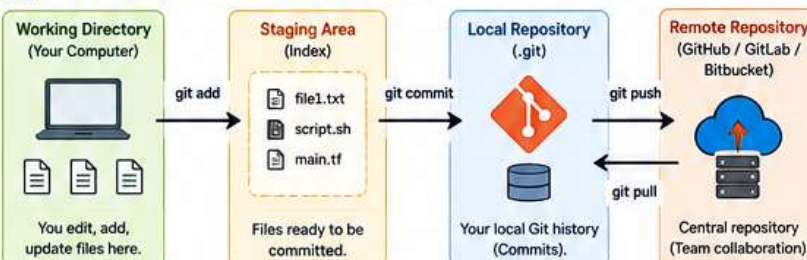
Version these files in Git:

- terraform/
- ansible/
- kubernetes/
- helm/
- scripts/
- .github/workflows/
- README.md
- CHANGELOG.md

Why version these?

- ✓ Team Collaboration
- ✓ Change Tracking
- ✓ Rollback
- ✓ CI/CD Automation
- ✓ Audit & Compliance
- ✓ Infrastructure as Code

12 GIT ARCHITECTURE DIAGRAM (DETAILED)



- git fetch : Downloads changes from remote but doesn't merge.
- git pull : Fetch + Merge changes from remote.

13 COMMIT ANATOMY

```
commit a1b2c3d4e5f6g7h8i9j0
Author: Narendra Pratap Singh <narendra@example.com>
Date: Sun May 12 18:30:45 2024 +0530

Update nginx configuration for better performance

Changed files:
modified: nginx.conf
modified: README.md
new file: scripts/deploy.sh
```

← Commit Hash (Unique ID)

← Author Info

← Commit Date

← Commit Message

← Changes in this commit



VERSION CONTROL IS THE FOUNDATION OF DEVOPS.
LEARN TODAY. AUTOMATE TOMORROW. SUCCEED ALWAYS.

KEEP LEARNING. KEEP BUILDING. LET'S GROW TOGETHER!



Better Collaboration



Time Travel
(History)



Safe & Secure



Faster Delivery



**Narendra
Pratap Singh**
DevOps & Cloud Engineer

VERSION CONTROL

DEEP DIVE - PART 1

Git Foundation for DevOps Engineers

★ Master the basics before moving to branching, release, CI/CD and production workflows.



SERIES ROADMAP

- 1 Git Foundation (You are here)
- 2 Branching & Merging
- 3 Collaboration & Workflows
- 4 Advanced Git Topics
- 5 Real-World Scenarios & Best Practices

1. WHAT IS VERSION CONTROL?

Version Control helps track every change in code, scripts, configs, infrastructure files and documentation.

For DevOps Engineer, it is used for:

- Application code
- Shell/Python scripts
- Terraform files
- Ansible playbooks
- Kubernetes manifests
- Helm charts
- CI/CD pipeline files
- Documentation

2. WHY DEVOPS ENGINEERS NEED GIT

- Track who changed what
- Rollback wrong changes
- Collaborate with team
- Maintain audit history
- Review changes before merge
- Automate CI/CD pipelines
- Version infrastructure as code

3. GIT ARCHITECTURE



- Working Directory:** Where you edit files.
- Staging Area:** Where you prepare files before commit.
- Local Repository:** Your local Git history.
- Remote Repository:** GitHub/GitLab/Bitbucket hosted repo.

4. BASIC GIT WORKFLOW



5. MOST USED GIT COMMANDS

```

01 git init
02 git clone <repo-url>
03 git status
04 git add .
05 git commit -m "message"
06 git log
07 git diff
08 git push
09 git pull
10 git fetch
  
```

6. GIT CONFIG COMMANDS

```

01 git config --global user.name "Narendra Pratap Singh"
02 git config --global user.email "your-email@example.com"
03 git config --list
  
```

7. REPOSITORY COMMANDS

```

01 git init
02 git clone <repo-url>
03 git remote -v
  
```

💡 Tip: Always verify remotes before push/pull.

8. DAILY WORKFLOW COMMANDS

```

01 git status
02 git add .
03 git commit -m "Updated DevOps notes"
04 git push origin main
05 git pull origin main
  
```

9. FILE STATE IN GIT



- Untracked:** New file not added to Git
- Modified:** Existing file changed
- Staged:** Ready for commit
- Committed:** Saved in Git history

10. BEST PRACTICES

- Commit small changes
- Write clear commit messages
- Pull before push
- Never commit passwords or secrets
- Use .gitignore
- Keep main branch stable
- Use branches for new work
- Review changes before commit

11. DEVOPS EXAMPLE

Version these files in Git:

- terraform/
- ansible/
- kubernetes/
- helm/
- scripts/
- .github/workflows/
- README.md
- CHANGELOG.md



12. .GITIGNORE EXAMPLE

```

# Dependency directories
node_modules/
.terraform/
.venv/

# Logs
*.log
npm-debug.log*

# OS generated files
.DS_Store
Thumbs.db
.idea/
.vscode/
  
```

13. FIRST TIME SETUP (ONE TIME)

```

git config --global user.name "Narendra Pratap Singh"
git config --global user.email "your-email@example.com"
git config --global core.editor code
git config --list
  
```

🛡️ Set once, use everywhere.
Keeps your identity in every commit.

Keep Learning.
Keep Building.
Let's Grow
Together! 🚀



LINKEDIN
linkedin.com/in/narendra09-devops



GITHUB
github.com/narendra09-devops



DEVOPS LEARNING HUB
<https://narendra09-devops.github.io/devops-learning-hub/>

★ Great DevOps Engineers build with code, automate with confidence, and deliver value every day! 🎯



**Narendra
Pratap Singh**

DevOps & Cloud Engineer

VERSION CONTROL DEEP DIVE – PART 1

Git Foundation for DevOps Engineers

Master the basics before moving to branching, release, CI/CD and production workflows. 🚀



linkedin.com/in/narendra09-devops



github.com/narendra09-devops



DevOps Learning Hub

https://narendra09-devops.

github.io/devops-learning-hub/



1 What is Version Control?

Version Control helps track every change in code, scripts, configs, infrastructure files and documentation.

For DevOps Engineer, it is used for:

- </> Application code
- > Shell / Python scripts
- 📁 Terraform files
- 📄 Ansible playbooks
- 📄 Kubernetes manifests
- 📄 Helm charts
- 🔗 CI/CD pipeline files
- 📄 Documentation

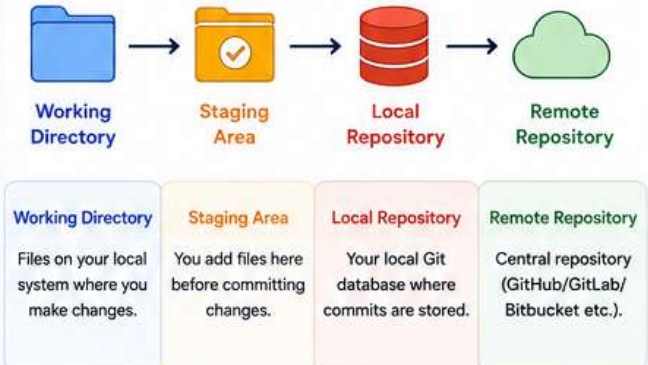
2 Why DevOps Engineers Need Git?

- ✅ Track who changed what
- ✅ Rollback wrong changes
- ✅ Collaborate with team
- ✅ Maintain audit history
- ✅ Review changes before merge
- ✅ Automate CI/CD pipelines
- ✅ Version infrastructure as code

💡 Git brings transparency, collaboration and confidence to every deployment!

3 Git Architecture

Understanding Git internals helps you use it like a pro.



4 Basic Git Workflow

- 1 Edit File
Make changes in your files
- 2 git status
Check what changed
- 3 git add .
Stage changes
- 4 git commit -m "message"
Commit with a meaningful message
- 5 git push origin main
Push to remote repository

5 Most Used Git Commands

```
# Start a repository
git init
# Clone a repository
git clone <repo-url>
# Check status
git status
# Add changes
git add .
# Commit changes
git commit -m "message"
# View commit history
git log
# See changes
git diff
# Push changes
git push
# Pull latest changes
git pull
# Fetch latest changes
git fetch
```

6 Git Config Commands

```
# Set your name
git config --global user.name "Narendra Pratap Singh"
# Set your email
git config --global user.email "your-email@example.com"
# Check all configs
git config --list
```

7 Repository Commands

```
# Initialize a new repo
git init
# Clone existing repo
git clone <repo-url>
# Check remote repositories
git remote -v
```

8 Daily Workflow Commands

```
git status
git add .
git commit -m "Updated DevOps notes"
git push origin main
git pull origin main
```

9 File State in Git

Files move through different states.



- Untracked : New file not added to Git
- Modified : Existing file changed
- Staged : Ready for commit
- Committed : Saved in Git history

10 Best Practices

- ✅ Commit small changes
- ✅ Write clear commit messages
- ✅ Pull before push
- ✅ Never commit passwords
- ✅ Use .gitignore
- ✅ Keep main branch stable
- ✅ Use branches for new work
- ✅ Review changes before commit

11 DevOps Example

Version these files in Git:

- 📁 terraform/
- 📁 ansible/
- 📁 kubernetes/
- 📁 helm/
- 📁 scripts/
- 📁 .github/workflows/
- 📄 README.md
- 📄 CHANGELOG.md

12 .gitignore Example

Common items to ignore:

```
# OS generated files
.DS_Store
Thumbs.db
# Logs
*.log
logs/
# Python
__pycache__/
*.pyc
# IDE
.vscode/
.idea/
# Terraform
.terraform/
*.tfstate*
# Secrets
.env
secrets/
```

13 Key Takeaways



Git is the backbone of DevOps and modern software delivery.



Master basics first, then move to advanced topics.



Good version control = Safe, reliable & auditable systems.



Small commits today, smooth deployments tomorrow!

**Stay Consistent.
Keep Learning. Keep Building.**
Let's Grow Together! 🚀



Version Control is the foundation of everything in DevOps. Master it. Own it. **Grow with it!**



Follow for more DevOps content and real-world practical guides.

